

Where a Decision Model Fits in an Agent Harness
Supplementary Material

Amplifier Michael J. Jabbour David Koleczek
Microsoft, Office of the CTO · October 7, 2026

Contents

A1 Evidence packages and provenance	2
A2 Decision-model benchmarks in detail	2
A3 The main paired campaign and the effort follow-ups	6
A4 The preregistered holdout study (S1)	7
A5 Production telemetry in detail	17
A6 Configuration recipes	18
A7 Review history and responses	20
A8 Glossary	20

Table S1: Evidence packages used in this report, by evidence package ID in the repository.

Evidence package ID	Contents	Status
2026-09-30-judge-benchmark	Constructed judge benchmark: development and holdout cases, label audit, latency	preregistered
2026-10-01-trace-judge-benchmark	Real-decision benchmark, 42 holdout cases	preregistered
2026-10-01-caching	Retrospective caching survey of earlier runs	exploratory
2026-10-02-paired-campaign	Main paired campaign, 1,036 sessions	preregistered
2026-10-04-clef-judges	Clef and Clef-Flash on all four judge splits	post hoc
2026-10-05-effort-control	Sonnet medium against default effort	designed in advance
2026-10-06-effort-control-fable	Fable medium against default effort	preregistered
2026-10-05-defaults-replay	Replay of the shipped routing code on recorded sessions	exploratory
2026-10-v3-offline	Telemetry analysis, decide-once counterfactual, projection inputs	exploratory
2026-10-06-holdout-v3	S1 holdout study, 1,224 sessions	preregistered
2026-10-06-openai-decisions	OpenAI Decisions API against an in-run Jev reference	post hoc

A1 Evidence packages and provenance

Every number in the paper and this supplement is regenerated from committed evidence packages by the build in the repository. [Table S1](#) lists them by their evidence package IDs in the repository; each carries its own data dictionary, checksums and, where preregistered, the preregistration and its provenance record.

Spend. The main campaign cost \$3,527.29, the effort follow-ups \$250.13 and \$421.71, S1 \$4,975.51, the judge benchmarks \$0.92 (constructed) and \$1.07 (real decisions), Clef \$0.21. Telemetry and the counterfactual analyses made no model calls.

A2 Decision-model benchmarks in detail

Constructed cases. 10 judges answered 90 development cases (a screen) and 63 holdout cases whose analysis was preregistered, 3 times each, under the harness’s gate (act only when the top probability is at least 0.90 and leads by 0.20, within 3 s). Two blind reviewer agents (language models, not people) checked the labels against a written guide; their agreement with the final labels was $\kappa = 1.00$, which shows the labels follow the guide, not that they match operator preference. [Figure S1](#) gives holdout accuracy for every judge, including the post hoc Clef run. No accuracy difference between Jev and the preregistered cloud judges was detected after the preregistered Holm correction; the paired bootstrap intervals for Luna and Sol (+1.6 to +15.9 and +3.2 to +19.0 points) exclude zero.

Real decisions. A second preregistered study rebuilt 42 held-out read decisions from earlier agent sessions by fingerprint matching, with the state clipped to the decision model’s input budget, and asked every judge again. The reviewer agents (again language models) agreed with each other ($\kappa = 0.84$) more than with what the host actually did next ($\kappa = 0.51$ – 0.57). The usefulness rule asked for at least 4 correct automatic reads with few misfires; 0 of 10 judges met it ([Figure S2](#)).

Risky actions: the side-effect interventions. Two interventions targeted wrong automatic actions with side effects. The deterministic guard (a short list of irreversible verbs matched on the proposed action) was confirmed on the holdout: it removed every side-effect wrong automatic action for all 9 applicable judges (38 to 0), at a coverage cost of 1.6–7.9 points. The natural-language risk

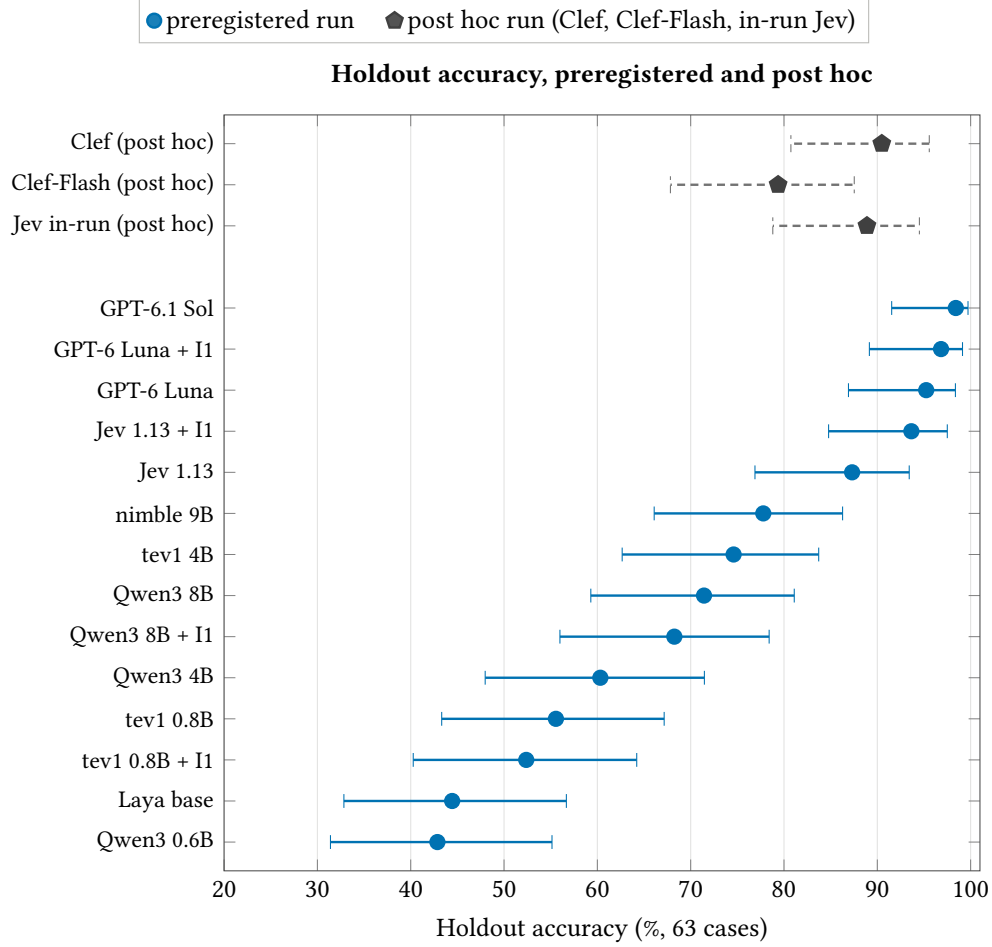


Figure S1: No accuracy difference between the top judges was detected after the preregistered correction. Holdout accuracy with 95 % Wilson intervals: the preregistered run, then (above the gap) the post hoc run of Clef, Clef-Flash and the in-run Jev reference.

How to read it. Each row is a judge; the dot is its accuracy and the whisker its 95 % interval. The top judges' whiskers overlap: no accuracy difference was detected after the preregistered correction (which is not the same as showing the judges equivalent). Jev sits among them at a fraction of the cost and latency of the cloud models.

clause ("Also choose reason if the prepared action would make an irreversible or external change (delete, purchase or pay, publish or post, send or submit, transfer, revoke, merge or deploy), even when the task requests it.") was not confirmed on the holdout (22 to 20 targeted errors; 1 of 3 applicable judges confirmed); on the development screen it removed all of Jev's side-effect errors (5 to 0).

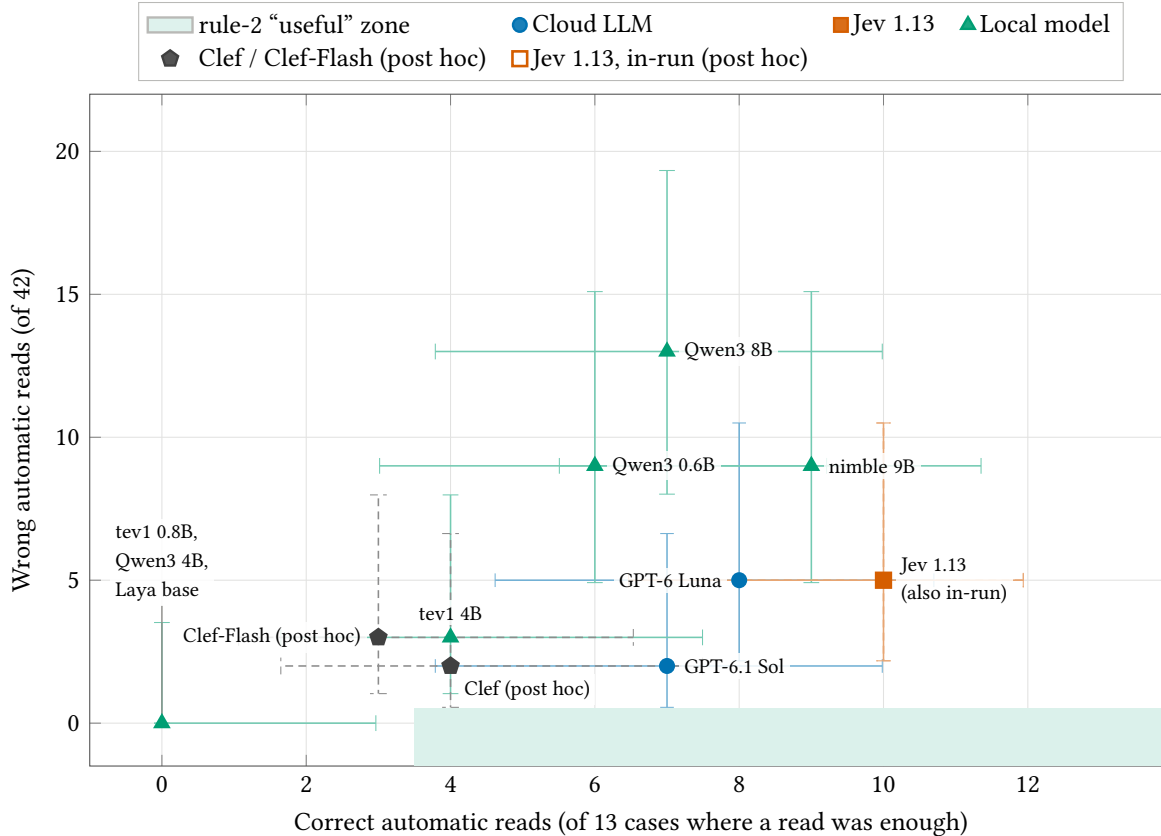


Figure S2: On real read decisions, no judge reached the usefulness zone. Real decisions: correct automatic reads against wrong automatic reads on trace-holdout, with 95 % Wilson intervals: the preregistered run, plus the post hoc Clef arms (black pentagons). The shaded strip is where the preregistered rule would call a judge useful.

How to read it. Right is more saved host calls; up is more misfires. No judge reaches the shaded strip: every judge with enough correct reads also misfired.

A2.1 OpenAI Decisions API (post hoc)

OpenAI’s Decisions API is a native endpoint for exactly this kind of typed question: GPT-6 Luna returns a probability per allowed answer. We ran it after the preregistered work, so everything here is post hoc and exploratory, against an in-run Jev 1.13 reference on the same cases: three repetitions in two option orders, the bundle’s gate with its 3 s deadline, and the per-case majority over repetitions (Table S2 and Figure S3).

- **Accuracy.** Luna was right on 88/90 development cases (Jev 86/90), 59/63 holdout cases (Jev 55/63), 16/21 real trace-dev decisions (Jev 16/21) and 30/42 trace-holdout decisions (Jev 23/42). Only the trace-holdout accuracy difference survived the Holm correction ($p = 0.039$; exploratory).
- **Misfires.** Luna’s wrong-automatic rate was no higher than Jev’s on every split; none of these differences was significant. Under the trace usefulness rule, neither judge was useful on either trace split.
- **Speed.** OpenAI’s servers reported 59–62 ms at the median and 136–179 ms at p95, but from our machine the client saw 1.4–1.8 s at the median and 3.0–4.8 s at p95. The network was

slow in this run for both providers: Jev’s client median was 1.4–1.6 s, against 0.15–0.17 s on 2026-10-04. The comparison between the two judges is paired in time; the absolute latencies are not comparable with earlier evidence.

- **Cost and reliability.** \$20–76 per million decisions against Jev’s \$16–46. In transport, 9 of 1,296 Luna requests and 4 of Jev’s failed.

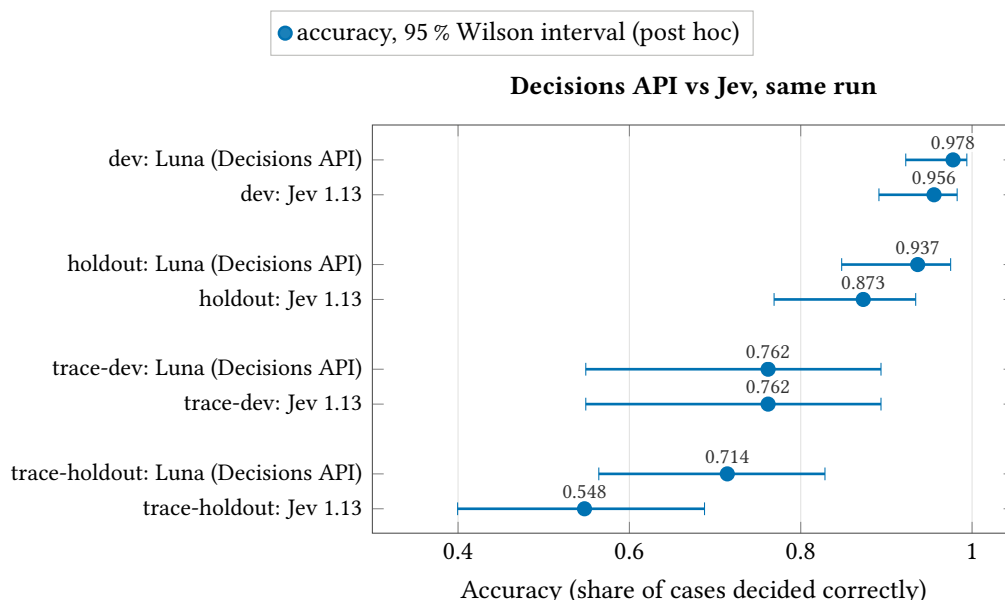


Figure S3: The Decisions API was at or above Jev on every split (post hoc). OpenAI Decisions API (GPT-6 Luna) against the in-run Jev reference: per-case majority accuracy with 95 % Wilson intervals on the four splits (post hoc).

How to read it. Each pair of rows is one split. Luna is at or above Jev on every split; only the trace-holdout gap survived the correction. Overlapping whiskers mean no difference was detected, not that the judges are equivalent.

Table S2: OpenAI Decisions API (post hoc) against the in-run Jev reference: correct decisions, wrong automatic actions, coverage, client latency (from our machine, slow network in this run) and dollars per million decisions.

Split	Judge	correct	wrong auto. (%)	coverage (%)	p50 s	p95 s	\$ / 1M
dev	GPT-6 Luna, Decisions API	88/90	0.0	71.1	1.80	4.80	20.3
dev	Jev 1.13 (in-run reference)	86/90	3.3	75.6	1.46	4.46	15.9
holdout	GPT-6 Luna, Decisions API	59/63	1.6	68.2	1.47	3.44	25.6
holdout	Jev 1.13 (in-run reference)	55/63	1.6	63.5	1.57	3.73	18.4
trace-dev	GPT-6 Luna, Decisions API	16/21	9.5	47.6	1.68	3.24	71.8
trace-dev	Jev 1.13 (in-run reference)	16/21	14.3	66.7	1.40	4.07	43.2
trace-holdout	GPT-6 Luna, Decisions API	30/42	2.4	16.7	1.41	3.00	76.2
trace-holdout	Jev 1.13 (in-run reference)	23/42	11.9	35.7	1.39	3.44	46.1

Key idea. A native typed-decision endpoint from OpenAI matched or exceeded Jev’s accuracy on our cases at 1.3–1.7× the price, but automated fewer real reads. Its servers answer in tens of milliseconds, but from here each call took seconds. It does not change the session-start finding: a one-line rule matched the decision model there.

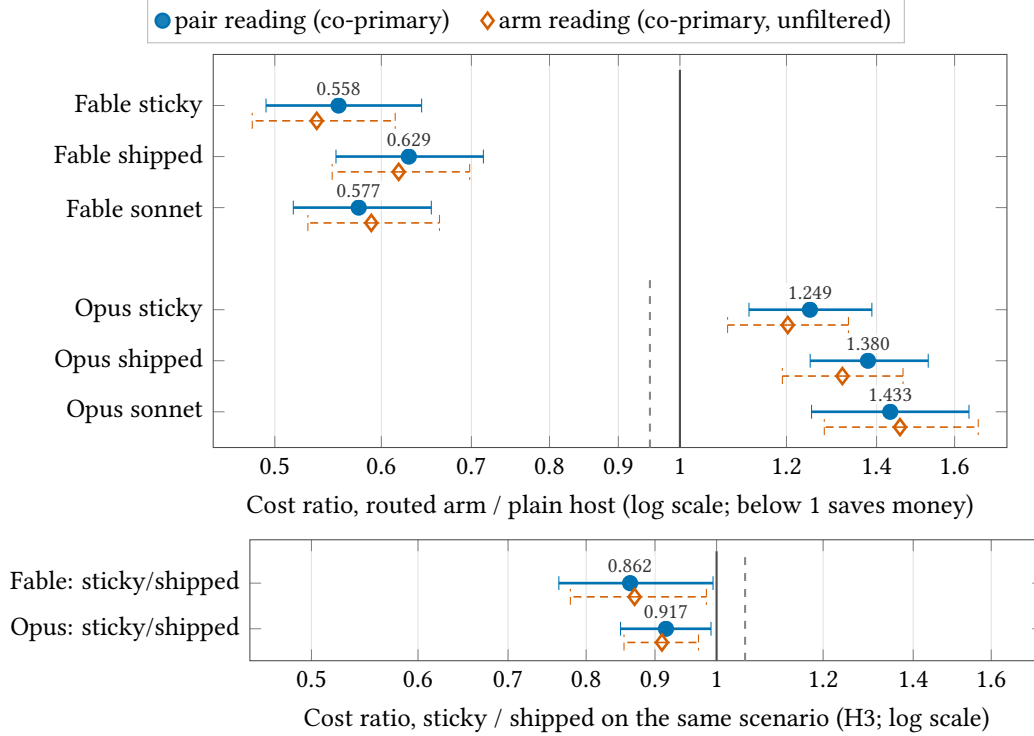


Figure S4: On the main-v1 test split, every Fable routing arm saved money and every Opus arm cost more. Confirmatory cost ratios on the main-v1 test split. Top: each routed arm against the plain host. Bottom: the sticky arm against the shipped arm on the same scenarios. Whiskers are 95 % intervals; the dashed lines mark preregistered thresholds.

How to read it. The dot is the geometric-mean cost ratio, the whisker its 95 % interval, on a log axis. Rows left of 1 saved money. All Fable rows are well left of 1; all Opus rows are right of it.

A3 The main paired campaign and the effort follow-ups

Design. The main campaign ran 1,036 sessions on 70 scripted scenarios (8–16 turns) on both hosts, in paired waves: every arm of a scenario started together from one frozen workspace, each with its own cache key. Hypotheses, thresholds and the train/test split were preregistered; the estimator was written after the data. Figure S4 gives the confirmatory test-split ratios. The pair reading conditions on quality and was written after the data; the arm reading uses every pair and agrees in direction.

Effort follow-ups. Two follow-ups measured reasoning effort directly, each on 23 test scenarios in concurrent pairs: plain Sonnet at medium cost $0.821 \times$ its default (0.781–0.861; designed in advance, with a provenance limitation recorded in its package) and plain Fable at medium cost $0.860 \times$ its default (0.833–0.885; preregistered), both with no detectable turn-pass loss. On Opus, medium effort did not demonstrate a saving in S1 ($0.992 \times$, 0.974–1.009).

Pricing every decide-once policy on the existing sessions. Because a decide-once policy's session equals either the plain-host session or the routed session of the same scenario, every such policy can be priced from the main campaign without new runs (Figure S5 and Table S3) [exploratory]. On Fable the one-line rule was at least as cheap as Jev; on Opus every routing policy cost more. The same re-analysis found the cache-rewrite cost of effort switches shown in the main paper.

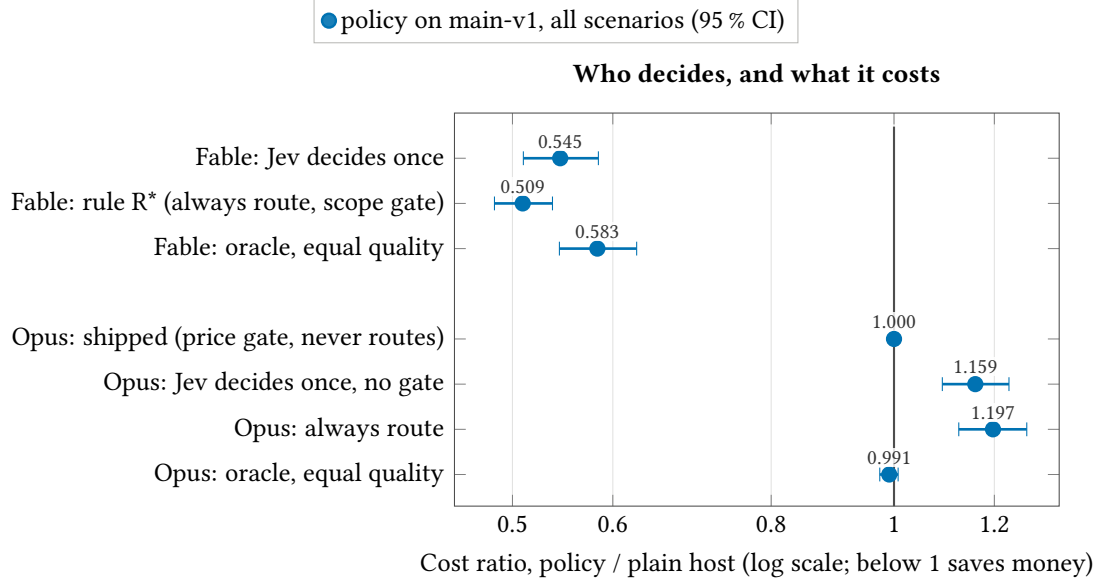


Figure S5: On Fable the rule R* is at least as cheap as Jev; on Opus every routing policy costs more. Decide-once policies priced on the existing main-v1 sessions (all scenarios, both reps), with 95 % scenario-cluster intervals. The oracle chooses with hindsight on the other rep, at equal quality. Exploratory.

How to read it. Each row is a policy. Left of 1 saves money. On Fable, the rule R* is at least as cheap as Jev; on Opus, every policy that routes costs more than keeping the host.

Table S3: Decide-once policies on main-v1 (exploratory): share routed, geometric-mean cost ratio and turn-pass difference against the plain host, and dollars per 1,000 sessions.

Policy (decided once per session)	routed	cost ratio [95 % CI]	turn-pass Δ [95 % CI]	\$ / 1,000
Fable: always host (plain)	0 %	1.000 [1.000, 1.000]	0.000 [0.000, 0.000]	5,311
Fable: Jev decides once	89 %	0.545 [0.510, 0.585]	-0.019 [-0.052, 0.010]	3,108
Fable: rule R* (always route, scope gate)	99 %	0.509 [0.484, 0.538]	-0.019 [-0.052, 0.010]	2,739
Fable: oracle, equal quality	81 %	0.583 [0.544, 0.627]	0.012 [-0.004, 0.030]	3,268
Opus: shipped (price gate, never routes)	0 %	1.000 [1.000, 1.000]	0.000 [0.000, 0.000]	2,125
Opus: Jev decides once, no gate	89 %	1.159 [1.092, 1.232]	-0.009 [-0.033, 0.013]	2,498
Opus: always route	100 %	1.197 [1.125, 1.273]	-0.005 [-0.030, 0.019]	2,620
Opus: oracle, equal quality	22 %	0.991 [0.975, 1.008]	0.000 [-0.011, 0.009]	2,124

A4 The preregistered holdout study (S1)

S1 tested the configurations frozen from the main campaign on 60 scenarios disjoint from it, smoke-tested before preregistration, with both hosts, two repetitions and a deterministic rule for choosing each host's configuration. This section gives every preregistered verdict and the figures behind the main paper's S1 numbers.

How the panel was prepared. Before the preregistration was committed, one plain-Sonnet smoke session ran on each of the 60 scenarios (\$182.80). It found prompt or grader defects in 30 scenarios: prompts that did not state what a check required, and checks that rejected correct answers. These were repaired; genuine agent mistakes were left as they were; every scenario was then revalidated mechanically (the starting workspace fails, the reference passes, wrong variants fail). The smoke results enter no hypothesis and are not pooled with S1.

S1 methods in brief. Quality. Each scripted turn carries automated checks (tests, file patterns, required facts or document sections); the *turn-pass fraction* is the share of a session’s turns whose checks passed, and a final-state check on the hidden tests is reported as additional evidence. The graders were validated before the run: the starting workspace must fail, the reference solution must pass every turn, and plausible wrong answers must fail. Non-inferiority means the 95 % lower bound of the turn-pass difference is above -0.05 , that is, no worse than the specified 5-point margin. These are claims about these automated checks, not about human judgement of, for example, the quality of a code review or an explanation.

Cost. The primary cost is the tools-normalized list-price cost of each session, plus estimated decider charges where a decider runs. The normalization reprices the first shared tool-definition prefix from the cache-write to the cache-read rate; it changed the accepted sessions’ recomputed cost by 0.45 % in main-v1 and 0.135 % in S1 (data dictionary in the evidence package).

Composed policies. S1 executed pinned-host, pinned-Sonnet and shipped-policy sessions. H1 and H2 evaluate decide-once policies by selecting the corresponding pinned outcome for each scenario and repetition: the rule or recorded Jev decision determines which session contributes its cost and quality. These policies therefore reuse executed outcomes rather than running a separate session for every configuration. Intervals resample whole scenarios. H6 separately compares live shipped sessions with their matched reconstructions, providing an empirical check of this cost decomposition (not of every quality property of every reconstructed policy).

Table S4: S1 (holdout-v3) preregistered hypotheses. Cost ratios are geometric means over scenarios; Holm within the two preregistered families. H3b is descriptive.

Hypothesis	cost ratio (95 % CI [†])	turn-pass Δ (95 % CI)	Holm p	verdict
H1	0.581 (0.542–0.625)	-0.013 (-0.028 to 0.002)	< 0.001	✓ supported
H2	1.008 (1.000–1.019) [†]	-0.003 (-0.008 to 0.000)	< 0.001	✓ supported
H3	0.999 (0.984–1.017) [†]	0.007 (-0.004 to 0.022)	< 0.001	✓ supported
H3b	1.004 (0.989–1.019) [†]	0.004 (-0.002 to 0.010)	–	descriptive
H4	0.992 (0.974–1.009)	-0.002 (-0.013 to 0.009)	0.187	✗ not supported
H5	1.255 (1.197–1.321)	-0.004 (-0.024 to 0.017)	< 0.001	✓ supported
H6	1.014 (0.996–1.032) [†]	0.004 (-0.011 to 0.019)	< 0.001	✓ supported
H7	0.616 (0.565–0.672)	-0.028 (-0.063 to 0.013)	0.120	✗ not supported

[†] H2, H3, H3b and H6 are equivalence tests; their cost interval is the 90 % interval the preregistration uses.

S1 ran 1,224 sessions on 60 scenarios. Table S4 lists the verdicts; in plain words:

- **H1 (Fable value), supported.** The frozen Fable configuration cost $0.581 \times$ plain Fable (95 % CI 0.542–0.625) with turn-pass -0.013 (-0.028 to 0.002). The saving is real, but smaller than predicted: the frozen prediction from main-v1 was 0.508 (0.483–0.536), and the observed ratio lies above that interval.
- **H2 (decision model against a rule), supported.** Letting Jev choose the session’s model and letting the one-line rule R^* choose it were equivalent: $1.008 \times$ (90 % CI 1.000–1.019), turn-pass

−0.003. They disagreed on only 4 of 120 scenario-reps. The preregistered decision follows: ship the simpler rule R^* , with no external call and no consent needed; Jev becomes an opt-in decider.

- **H3 (bundle overhead), supported.** The shipped bundle on Opus cost $0.999\times$ plain Opus (90 % CI 0.984–1.017): cost-equivalent within the preregistered ± 5 % margin. On Fable, the bundle with the host pinned cost $1.004\times$ (descriptive).
- **H4 (Opus at medium effort), not supported.** Opus at medium cost $0.992\times$ its default (95 % CI 0.974–1.009): medium effort did not demonstrate a saving, unlike Sonnet ($0.821\times$) and Fable ($0.860\times$).
- **H5 (routing on Opus), supported.** Routing to Sonnet still lost on Opus: $1.255\times$ (1.197–1.321), on fresh scenarios and without the effort-switch artefact.
- **H6 (live equals predicted), supported.** Live decide-once sessions cost what the decomposition predicted ($1.014\times$, 90 % CI 0.996–1.032).
- **H7 (review and explain on Sonnet), not supported.** On the 20 review and explain scenarios (19 for cost, after the gate-failed scenario is excluded), the turn-pass lower bound for Sonnet against the host was -0.063 , below the -0.05 margin: non-inferiority was not established (the interval also includes zero, so a loss is not shown either). The preregistered rule therefore keeps these task types on the host (`keep_on_host: [review, explain]`).

What the freeze rule chose. On Fable the rule picked the configuration ($0.528\times$)

`always_route | strong default | scope off | keep none`

not the preregistered

`rstar | strong medium | scope 300 | keep none`

That makes it *selected on holdout*: it must replicate in a follow-up study before it ships. It also conflicts with H7, which keeps review and explain on the host. What ships for Fable until S2 replicates is therefore the preregistered configuration plus `keep_on_host: [review, explain]`, applied through a keyword proxy (main paper, Section 8). On Opus no candidate qualified, so Opus keeps the shipped defaults: no routing, default effort.

Key idea. The decision model’s start-of-session choice was matched by a one-line rule on these hosts and prices. Where the money is, routing on Fable and never on Opus, price arithmetic and a rule decide it. The value of a system-1 model is not in this decision.

Process note: how the S1 run went. The study spent \$4,975.51. 8 Fable sessions of one scenario (a code-review scenario) were served partly by Opus, most likely the provider's fallback (an inference); the preregistered rule excluded them from cost, so Fable cost contrasts use 59 scenarios. The stop rule for gate failures was not followed to the letter, because the gate is evaluated when rows are extracted, not live; this is a disclosed deviation. A Forge outage made 135 of 376 wave attempts (36 %) fail on infrastructure; every failed wave was rerun whole. The budget cap was raised from \$5,000 to \$6,300 during the run, though the final spend stayed under the original cap, and concurrency changed several times. The A/A copies differed in turn-pass by -0.047 on Opus (11 pairs), so small quality differences cannot be interpreted. Rerunning the analysis without the flagged scenarios changed no verdict and no freeze choice.

A4.1 Every hypothesis on one page

Figure S6 puts all eight preregistered comparisons side by side, cost on the left and quality on the right, so the reader can see at once which results move money and which are guards.

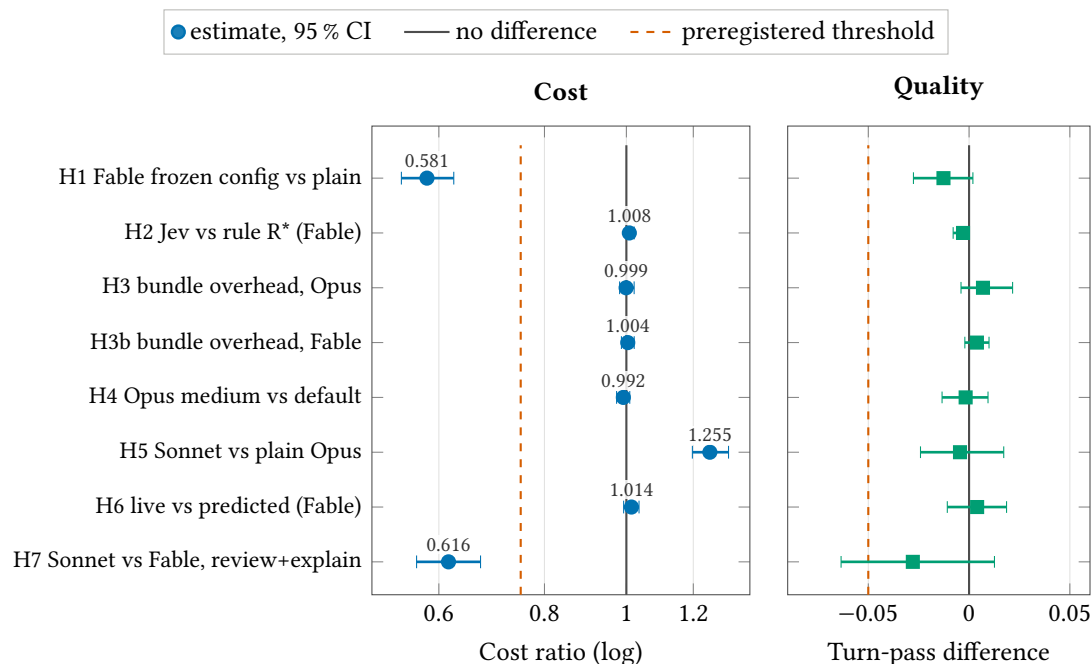


Figure S6: Two hypotheses carry money (H1, H5); the rest are guards. S1 hypotheses H1–H7: cost ratio (left, log scale) and turn-pass difference (right), each with its 95 % interval. H3b is descriptive. Thresholds: the H1 cost bound (0.75) and the quality margin (-0.05).

How to read it. Read each row across both panels. A row helps a configuration when its cost dot is left of 1 and its quality whisker stays right of the dashed line. H1 does both. H5 is far right of 1: routing on Opus costs more. H4 straddles 1: medium effort did nothing measurable on Opus. H7's quality whisker crosses the dashed line, which is why review and explain stay on the host.

Key idea. Two of the eight rows carry money: H1 (Fable routing saves about two fifths) and H5 (Opus routing loses about a quarter). The rest are guards: the bundle is cost-equivalent within the $\pm 5\%$ margin (H3), the model adds nothing over the rule (H2), live sessions behave as predicted (H6), and review and explain need protection (H7).

A4.2 How to read a paired cost ratio

Worked example: one scenario, two reps, one ratio. Take one explain scenario from the holdout study, the scenario whose ratio sits closest to the overall result. Under the frozen Fable configuration it was routed to Sonnet. Across its 2 repetitions the routed sessions cost \$2.34 on average and the plain Fable sessions \$4.04. Its paired cost ratio is therefore $2.34/4.04 = 0.580$; on the log scale that is $\ln 0.580 = -0.544$. Doing the same for each of the 59 scenarios, averaging the logs and exponentiating gives the *geometric mean*, 0.581: the H1 headline. The interval comes from resampling whole scenarios 10,000 times and repeating the calculation.

Why a geometric mean? A ratio of 0.5 and a ratio of 2 should cancel; their arithmetic mean (1.25) says “more expensive”, their geometric mean (1) says “no change”. Averaging logs treats halving and doubling symmetrically.

A4.3 The variance underneath the headline

A geometric mean hides how much individual tasks vary. Figure S7 shows every scenario’s paired ratio, sorted, for the two results that carry money.

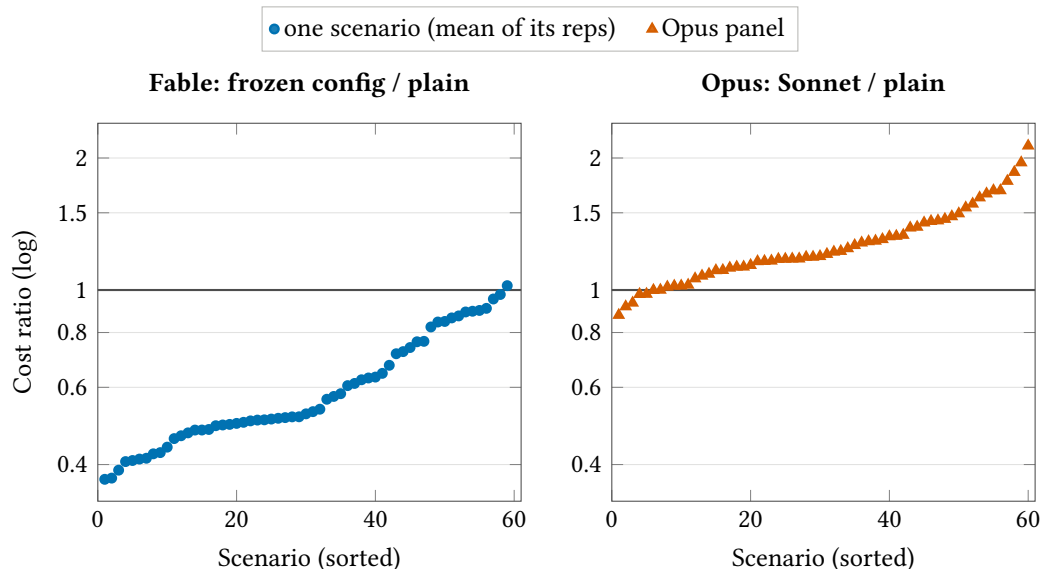


Figure S7: The direction is consistent across scenarios; the size varies widely. Per-scenario paired cost ratios in S1, sorted. Left: the frozen Fable configuration against plain Fable (59 scenarios). Right: Sonnet decided once against plain Opus (60 scenarios). Log scale.

How to read it. Each dot is one scenario. Below the line saves money. On Fable, 1 of 59 scenarios cost more than plain Fable, and the ratios run from 0.37 to 1.02. On Opus, 53 of 60 cost more, from 0.88 to 2.13. The direction is consistent; the size varies a lot from task to task.

A4.4 Where the risk sits: task types and families

The quality cost of routing is not spread evenly. Figure S8 splits the Fable result by task type and by scenario family (exploratory).

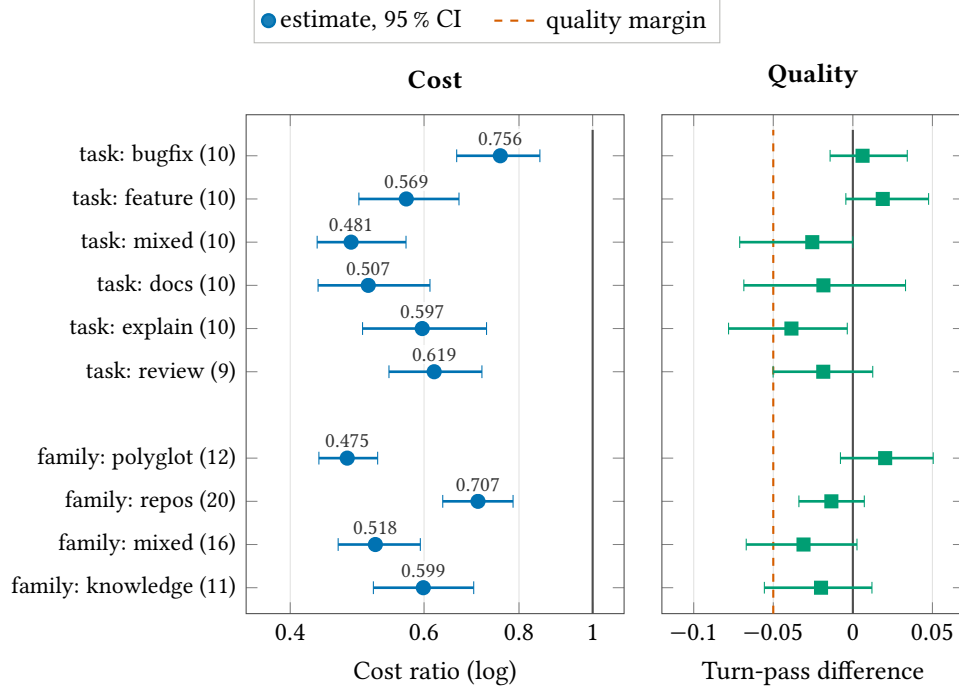


Figure S8: Every subgroup saves money; quality intervals for several subgroups are too wide to rule out a loss. Lower bounds fall below the -0.05 margin for the explain (-0.078), mixed (-0.071) and docs (-0.069) tasks and the mixed (-0.067) and knowledge (-0.056) families; review’s lower bound is -0.050 . Subgroups hold 9–20 scenarios; treat as hypothesis-generating. The frozen Fable configuration against plain Fable, split by task type and by scenario family (number of cost-valid scenarios in brackets; the quality panel is unfiltered and uses every scenario of the subgroup), with 95 % intervals. Exploratory: subgroups were not preregistered except H7.

How to read it. Every subgroup saves money. The quality panel is where they differ: explain (-0.039 ; lower bound -0.078) and mixed work cross the margin, review touches it (-0.050), while bugfix and feature sit near zero. Small subgroups give wide whiskers; treat single rows with caution.

A4.5 Which configuration the rule picked

The freeze rule considered every configuration the preregistration listed and chose one per host. [Figure S9](#) shows them all, with the choice and the preregistered configuration marked; [Table S6](#) lists them.

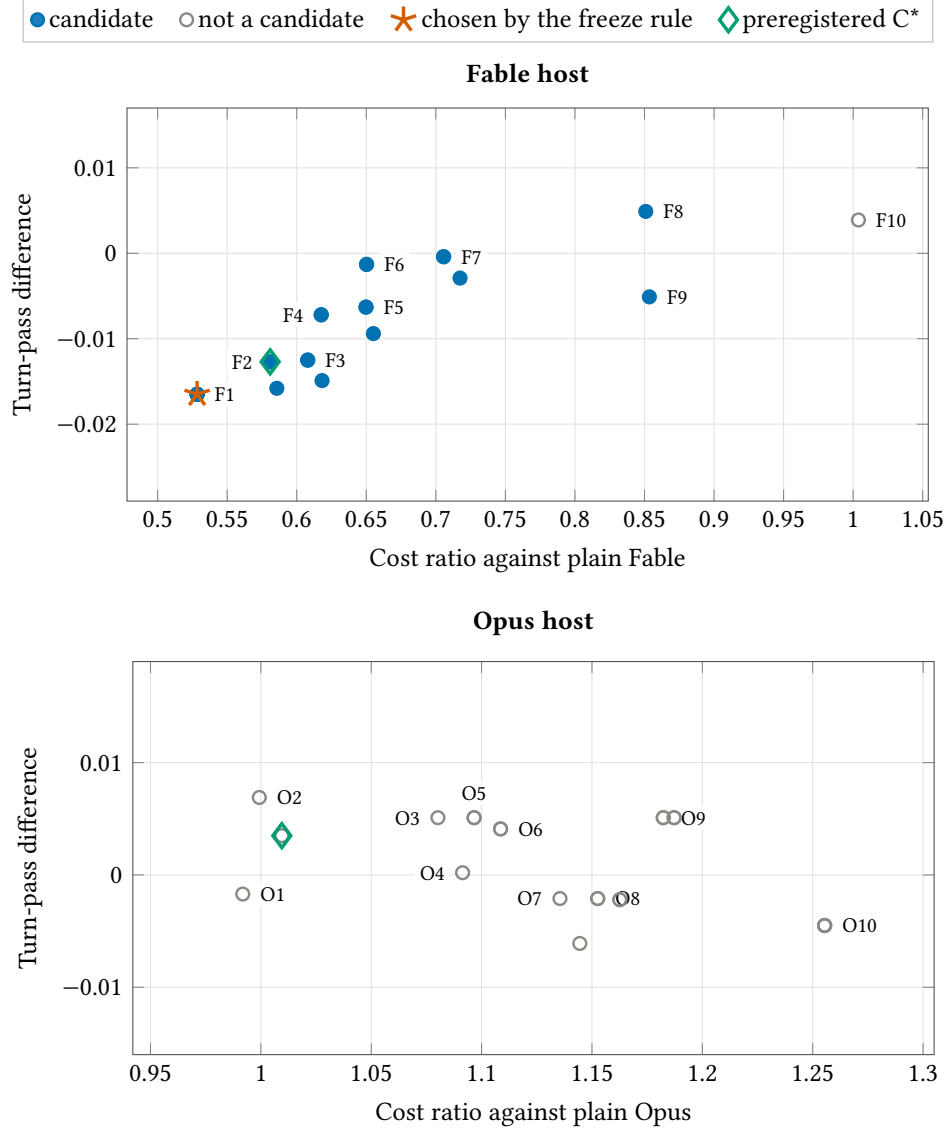


Figure S9: On Fable many configurations qualify; on Opus none does. All 23 Fable and 23 Opus configurations the freeze rule considered: cost ratio against the host’s plain default and turn-pass difference (point estimates). Keys group configurations that land on the same point; Table S6 lists them.

How to read it. Left and up is better. Filled dots passed the preregistered candidate test (cost upper bound below 1, quality lower bound above the margin, final-state point no worse than the specified limit). The star is the rule’s choice; the diamond is the preregistered configuration. On Fable 22 configurations qualified; the cheapest cluster drops the scope gate. On Opus none qualified, so the shipped default stays.

A4.6 The decision model against the rule

Jev and the rule R* chose differently on only 4 of 120 Fable scenario-reps (Table S5).

Table S5: The 4 S1 scenario-reps on which the decision model (Jev) and the rule R* chose different session models, with what each choice cost and how it scored.

Scenario	rep	model chosen by		session cost		turn-pass	
		Jev	rule R*	Jev	R*	Jev	R*
go-bowling	1	Fable, medium	Sonnet (Pc)	\$6.53	\$4.06	0.857	1.000
go-bowling	2	Fable, medium	Sonnet (Pc)	\$5.66	\$4.77	0.929	0.929
peewee-filterfk	1	Fable, medium	Sonnet (Pc)	\$2.15	\$1.94	0.889	1.000
peewee-filterfk	2	Fable, medium	Sonnet (Pc)	\$2.34	\$1.87	0.889	1.000

Worked example: why the two deciders cannot differ much. Two deciders that agree on a scenario-rep produce the same session there, so their difference comes only from the scenario-reps where they disagree. The share of those is $d = 4/120 = 0.033$. On them the choices differed in cost by \$1.01 per session on average. So the most the deciders can differ, averaged over all sessions, is $d \times \$1.01 = \0.034 per session, about 1.2 % of a typical session (\$2.87). That bound, not a significance test, is what makes “equivalent” believable: with so few disagreements, no amount of skill on those few can move the total.

A4.7 Effort, across three hosts

Three separate studies measured medium against default reasoning effort, one per host model. Figure S10 puts them on one chart.

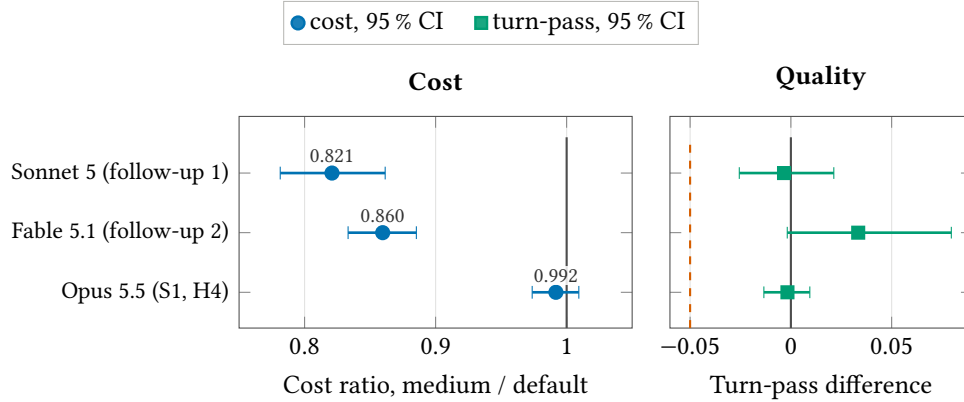


Figure S10: Medium effort saved money on Sonnet and Fable but did not demonstrate a saving on Opus. Medium against default reasoning effort, measured in three studies: Sonnet (follow-up 1), Fable (follow-up 2) and Opus (S1, H4). 95 % intervals.

How to read it. Sonnet and Fable sit well left of 1: medium effort is cheaper with no detectable quality loss. Opus sits on 1. The same setting saves money on two models and nothing on the third, so effort must be set per host.

A4.8 Where the money goes

Why does the same switch save money on one host and cost money on the other? Figure S11 splits the average S1 session’s cost by token class.

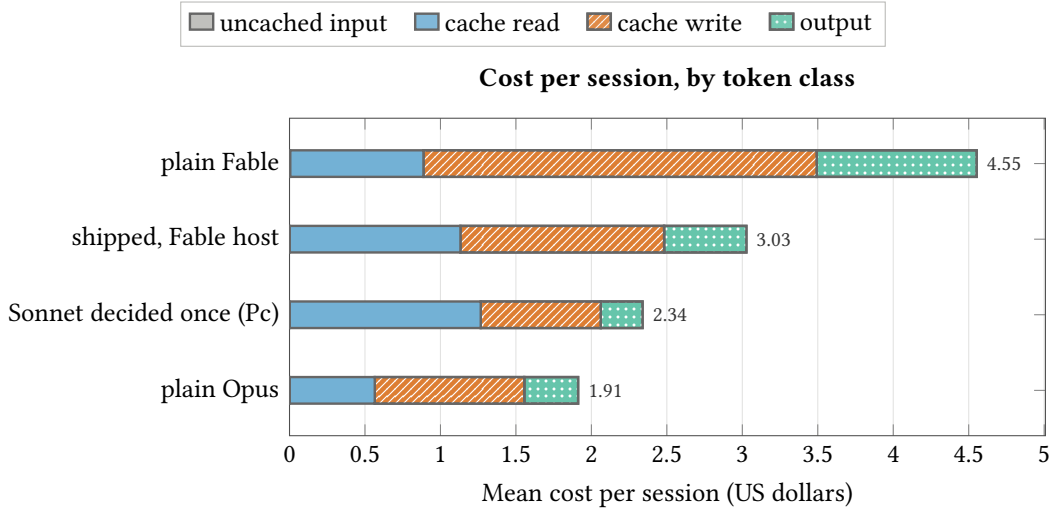


Figure S11: Plain Fable spends most of its money writing to the cache. Mean cost per S1 session split by token class, at the study’s price table (exploratory).

How to read it. Each bar is the average session. Plain Fable spends most on cache writes (57 % of \$4.55): writing context into Fable’s cache is expensive. The Sonnet session (\$2.34) spends more on cache reads (54 %), which are cheap per token but many. Plain Opus (\$1.91) is already cheaper than the Sonnet session, which is the whole H5 story in one bar.

A4.9 Every configuration the freeze rule considered

Table S6: Every configuration the S1 freeze rule considered (keys as in Figure S9). Cost ratio and turn-pass difference against the host’s plain default.

Key	Host	Configuration	cost ratio	turn-pass Δ	candidate
F1	Fable	always_route strong default scope off keep none	0.528	−0.017	✓
F1	Fable	always_route strong medium scope off keep none	0.528	−0.017	✓
F1	Fable	rstar strong default scope off keep none	0.528	−0.017	✓
F1	Fable	rstar strong medium scope off keep none	0.528	−0.017	✓
F2	Fable	always_route strong medium scope 300 keep none	0.581	−0.013	✓
F2	Fable	rstar strong medium scope 300 keep none	0.581	−0.013	✓
F2	Fable	jev strong medium scope 300 keep none	0.586	−0.016	✓
F3	Fable	always_route strong default scope 300 keep none	0.608	−0.012	✓
F3	Fable	rstar strong default scope 300 keep none	0.608	−0.012	✓
F3	Fable	jev strong default scope 300 keep none	0.618	−0.015	✓
F4	Fable	always_route strong medium scope off keep review+explain	0.618	−0.007	✓
F4	Fable	rstar strong medium scope off keep review+explain	0.618	−0.007	✓
F5	Fable	always_route strong medium scope 300 keep review+explain	0.650	−0.006	✓

Key	Host	Configuration	cost ratio	turn-pass Δ	candidate
F5	Fable	rstar strong medium scope 300 keep review+explain	0.650	-0.006	✓
F5	Fable	jev strong medium scope 300 keep review+explain	0.655	-0.009	✓
F6	Fable	always_route strong default scope off keep review+explain	0.650	-0.001	✓
F6	Fable	rstar strong default scope off keep review+explain	0.650	-0.001	✓
F7	Fable	always_route strong default scope 300 keep review+explain	0.706	0.000	✓
F7	Fable	rstar strong default scope 300 keep review+explain	0.706	0.000	✓
F7	Fable	jev strong default scope 300 keep review+explain	0.717	-0.003	✓
F8	Fable	plain-medium	0.851	0.005	✓
F9	Fable	bundle-unrouted medium	0.853	-0.005	✓
F10	Fable	bundle-unrouted default	1.004	0.004	
O1	Opus	plain-medium	0.992	-0.002	
O2	Opus	bundle-unrouted default	0.999	0.007	
O2	Opus	bundle-unrouted medium	1.009	0.003	
O3	Opus	jev strong default scope 300 keep review+explain	1.080	0.005	
O4	Opus	jev strong medium scope 300 keep review+explain	1.091	0.000	
O5	Opus	always_route strong default scope 300 keep review+explain	1.097	0.005	
O5	Opus	rstar strong default scope 300 keep review+explain	1.097	0.005	
O6	Opus	always_route strong medium scope 300 keep review+explain	1.109	0.004	
O6	Opus	rstar strong medium scope 300 keep review+explain	1.109	0.004	
O7	Opus	jev strong default scope 300 keep none	1.136	-0.002	
O7	Opus	jev strong medium scope 300 keep none	1.144	-0.006	
O8	Opus	always_route strong default scope 300 keep none	1.153	-0.002	
O8	Opus	rstar strong default scope 300 keep none	1.153	-0.002	
O8	Opus	always_route strong medium scope 300 keep none	1.163	-0.002	
O8	Opus	rstar strong medium scope 300 keep none	1.163	-0.002	
O9	Opus	always_route strong default scope off keep review+explain	1.182	0.005	
O9	Opus	rstar strong default scope off keep review+explain	1.182	0.005	
O9	Opus	always_route strong medium scope off keep review+explain	1.187	0.005	
O9	Opus	rstar strong medium scope off keep review+explain	1.187	0.005	
O10	Opus	always_route strong default scope off keep none	1.255	-0.004	
O10	Opus	always_route strong medium scope off keep none	1.255	-0.004	
O10	Opus	rstar strong default scope off keep none	1.255	-0.004	

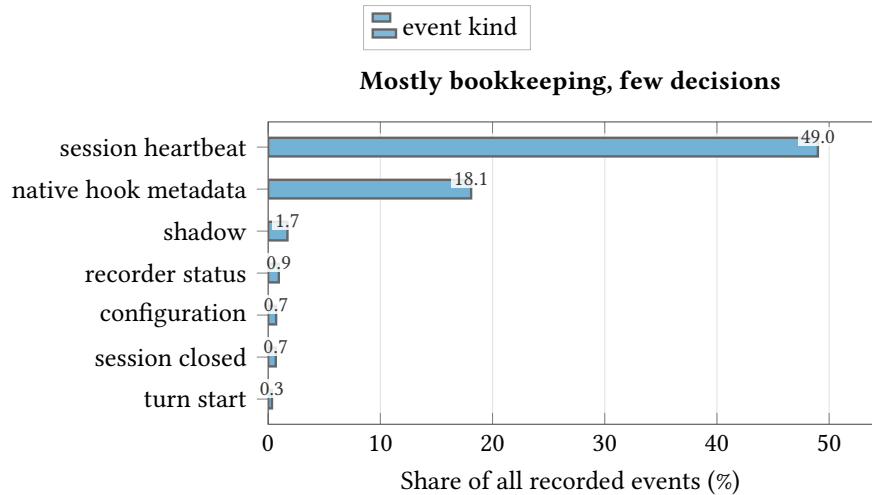


Figure S12: Most recorded events are bookkeeping; decision-bearing events are a thin slice. What the observatory store holds: the seven most common kinds of recorded event, as a share of all events.

How to read it. The longest bar, session heartbeat, is 49.0 % of everything recorded. Decision-bearing events are a thin slice. A store this dominated by bookkeeping can tell you that decisions happen, not whether they were right.

Key	Host	Configuration	cost ratio	turn-pass Δ	candidate
O10	Opus	rstar strong medium scope off keep none	1.255	−0.004	

A5 Production telemetry in detail

The telemetry window ran from 2026-09-17 to the 2026-10-05 snapshot (19 days, 464,524 events). Of the 398 production sessions, 77 carried an explicit production tag and 321 were classified as production because they carried no test tag. [Figure S12](#) shows what the store holds.

The 7.1 % that was not what it seemed. The telemetry reported that the read-shortcut proposals agreed with the host’s next action only 7.1 % of the time (117 of 1,642). Taken at face value, that condemns the idea. Decomposed, it says little about the decision model:

- 1,491 of the 1,642 decisions followed a host call to a tool the proposal could not contain, so they could never match. “Agreement” was partly a labelling artefact.
- 648 were scored by a synthetic demo backend that always picks the first candidate.
- Where a match was possible, the real local scorer agreed with the host 77.5 % of the time (117 of 151; 95 % CI 70.2–83.4 %).

Agreement is still not correctness: the store keeps no state text, so whether a disagreement was a mistake cannot be judged from it.

Claimed savings were estimates. The bundle’s own receipts claimed \$222.66 saved by keeping caches warm (194 production receipts) and showed per-step routing to Sonnet costing money (the receipts summed to −1.80 dollars). The read shortcut “would have” avoided 117 host turns in 994 decisions. None of these was measured as a saving; later measurement turned the per-step routing off. In production, the first start-tier decision of a session was most often the scope gate keeping the host (105 of 214 sessions, 49 %); Jev judged 68 sessions strong and 35 cheap.

The corrected projection. Before S1, a provisional projection combined this owner's recorded spend mix (53 % Opus, 46 % Sonnet) with per-host ratios and gave $0.83\times$ (0.80–0.86) of shipped defaults. It used a prior of 0.821–0.860 for Opus at medium effort, taken from the Sonnet and Fable effort studies. S1 tested that directly and medium effort did not demonstrate a saving on Opus ($0.992\times$). With Opus at default effort and Sonnet hosts at medium (0.821, 0.781–0.861), the same mix gives $0.92\times$ (0.90–0.94). Neither figure meets the program's cost target of $0.50\times$. No production session ran on Fable, so the configuration's main saving has no production exposure yet.

A6 Configuration recipes

The recipes below use only keys that exist in the shipped bundle (version 0.3.0). The first listing is the orchestrator configuration exactly as it ships; the others change one or two keys of it.

The shipped default (Fable, Opus and Sonnet hosts alike). The same configuration serves every host: the price gate keeps Opus sessions on Opus, `by_host` sets Fable's own effort to medium, and `keep_on_host` keeps review- and explain-like sessions on the host.

```
config:
  mode: active
  backend: jev
  allow_external_state: false
  read_shortcut: false
  timeout_ms: 3000
  allowed_tools: [fast_workspace]
  effort_routing:
    by_tier:
      cheap: medium
      strong: null
    by_host:
      claude-fable-5-1:
        strong: medium
  model_routing:
    start_model: claude-sonnet-5
    provider_match: anthropic
    start_policy: rules
    complex_min_prompt_chars: null
    decision_scope: session
    price_gate:
      enabled: true
    keep_on_host:
      task_types: [review, explain]
    cheap_max_workspace_files: 300
    max_requests_before_escalation: null
    escalate_on_test_failure: false
    escalate_on_provider_error: true
  cache_keepalive:
    enabled: true
    interval_s: 270
    max_refreshes: 6
    min_prefix_tokens: 20000
    max_output_tokens: 1
  waste_guards:
    enabled: true
    repeat_stop: true
    retry_stop: true
    poll_wait: true
    identical_results: true
    poll_max_wait_s: 240
```

```
observatory:
  enabled: true
  open_browser: always
  port: 8765
```

Recipe: accept the review and explain risk for the larger saving (Fable). Remove the opt-out. Expected cost falls from about 0.75× to about 0.58× of plain Fable, at the quality risk H7 measured.

```
model_routing:
  keep_on_host:
    task_types: []
```

Recipe: medium effort on a Sonnet host. Sonnet at medium cost 0.821× its default. The per-host map takes any model id:

```
effort_routing:
  by_host:
    claude-sonnet-5:
      strong: medium
```

Do not add Opus to this map: medium effort did not demonstrate a saving on Opus (0.992×).

Recipe: use the decision model instead of the rule. Jev chose the same session model as the rule on all but 4 of 120 scenario-reps, so this buys nothing measurable and sends the starting state to an external service. If you want it anyway, opt in explicitly:

```
allow_external_state: true
model_routing:
  start_policy: judge
```

Claude Code, Codex and Copilot CLI. The smart tool makes the same session-start decision outside the harness. `decide` prints the choice; `launch` makes it and starts the third-party agent with that model and effort (`--dry-run` prints the command instead):

```
npm skills add michaeljabbour/amplifier-bundle-fast-decisions
amplifier-fast-decisions decide \
  --task "Fix the failing date parser test" \
  --host-model claude-fable-5-1 --workspace . --decider rules
amplifier-fast-decisions launch --harness claude \
  --task "Explain the cache layer" \
  --host-model claude-fable-5-1 --workspace . --dry-run
amplifier-fast-decisions launch --harness codex \
  --task "..." --host-model claude-fable-5-1
amplifier-fast-decisions launch --harness copilot \
  --task "..." --host-model claude-fable-5-1
npm skills update
```

Inside the Amplifier harness itself, add the bundle and keep it current with `amplifier bundle update`. Savings outside the harness have not been measured yet.

A7 Review history and responses

The study was reviewed in three rounds. Rounds one and two (an external reviewer) concerned the main campaign: the estimator written after the data, the quality filter, effort as a confound and the measurement of quality; every point was answered in the companion technical report, and two follow-up effort studies were run in response. Round three (an independent reviewer) reran the S1, main-campaign and effort analyses from the published rows, found them reproducible, and asked for focused corrections, all applied:

1. **Row labels reversed in four figures.** Fixed; every bar and forest chart now prints each row's value beside it, and a build-failing check compares every rendered row label and value with its source table.
2. **Statistical wording.** H7 reads "non-inferiority not established"; judge accuracy reads "no accuracy difference was detected after the preregistered correction" (not equivalence; H2 remains a true equivalence test); H3 reads "cost-equivalent within the preregistered $\pm 5\%$ margin"; Opus effort reads "medium effort did not demonstrate a saving".
3. **Holdout and telemetry descriptions.** The S1 panel is described as disjoint from the main campaign and smoke-tested before preregistration, with the repair account ([Section A4](#)); the telemetry window is stated in days.
4. **S1 methods.** Quality, cost and the composed policies are defined ([Section A4](#)), with H6 as the empirical check of the cost reconstruction and separate cost and quality denominators.
5. **Effort scale.** A cross-study log-cost share was replaced by the direct follow-up result.
6. **Measurement and rebuild.** The tool-definition normalization is defined and sized (0.45 % in the main campaign, 0.135 % in S1), and every input read from version control is pinned to a commit.
7. **Related work, editorial points and author block.** Applied.

A8 Glossary

A/A pair

Two identical plain-host sessions; their difference shows the noise floor.

Cache read / cache write

Tokens served from, or written into, the provider's prompt cache. Reads are cheap; writes cost more than ordinary input.

Confidence interval (95 %)

An interval produced by a procedure that, over many repetitions of the study, would contain the true value of the parameter 95 % of the time; here computed by resampling whole scenarios.

Decide once

Choose the session's model and effort at the start and never switch.

Decision model (system 1)

A fast model that answers a typed question with a probability per allowed answer.

Freeze rule

The preregistered, deterministic rule that picks each host's configuration from the S1 results.

Geometric mean

The average of logs, exponentiated; the right average for ratios.

Holm correction

A way to keep the overall false-positive rate at 5 % when testing several hypotheses.

Host model

The large model the agent normally runs on (Fable 5.1 or Opus 5.5 here).

keep_on_host

A setting that keeps chosen task types (review, explain) on the host model.

Non-inferiority margin

How much worse quality may be and still count as “no worse than the specified margin”: 5 points of turn-pass.

Paired cost ratio

One arm’s session cost divided by its partner’s, on the same task, started together.

Preregistered

Hypotheses and thresholds fixed in a committed document before the data existed.

Price gate

Arithmetic that keeps the host when the cheaper model cannot save money at current prices.

Rule R*

Route to Sonnet unless the workspace has more than 300 files.

Scope gate

The workspace-size part of R*.

Turn-pass fraction

The share of a session’s scripted turns whose checks passed.